SASL Spring 2026 Research Project

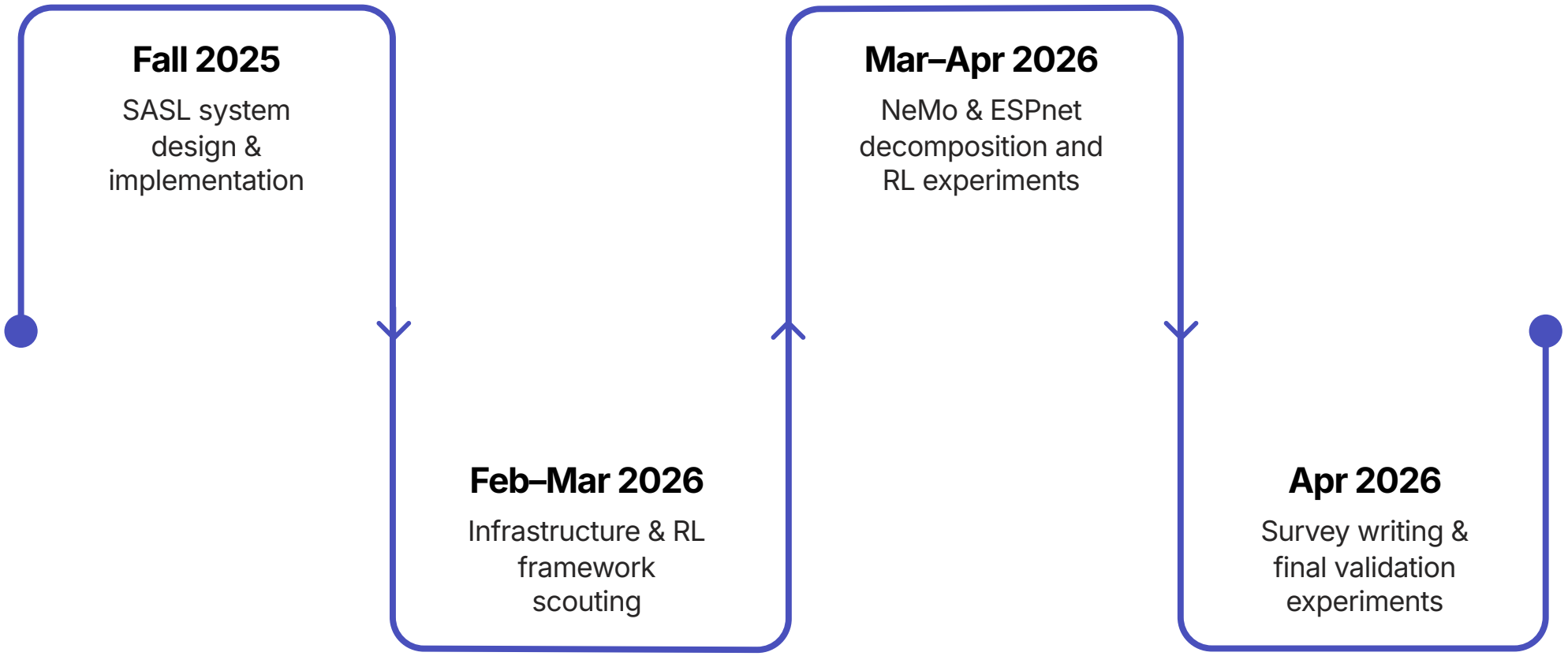
Adaptive Self-Learning STT and Survey of RL Tooling and Fine-Tuning Frameworks

Gautam Agarwal, Columbia University
Shivangi Kumar, Columbia University
Chen Wang, IBM Research Center

Started as COMS 6998 course project in Fall 2025

Extended into research and survey in Spring 2026

Project Timeline



Our work progressed from core system design to infrastructure, framework evaluation, model integration, and final synthesis. Each milestone built directly on the previous stage and moved the project toward a complete adaptive self-learning STT pipeline and tooling survey.

Motivation: Self-Adaptive STT

THE PROBLEM

- Real-world STT deployments face domain shift, noisy acoustics, and specialized vocabulary
- Accuracy degrades over time relative to benchmarks in production environments

CONVENTIONAL APPROACH

Static pretrain + occasional manual retraining

- Expensive human labeling and high GPU costs
- Reactive rather than adaptive

OUR THESIS

- Close the loop: detect errors → auto-correct with LLM → adaptively fine-tune
- All within a compute budget constraint
- Enables continuous improvement without manual intervention

SASL Results From Course Project & Why We Pivoted

KEY RESULTS

20.44%

Baseline WER

9.26%

Baseline CER

17.25%

Full System WER

Edinburgh Noisy Speech

7.56%

Full System CER

Edinburgh Noisy Speech

- Error detection + LLM correction delivered the largest WER reduction
- Adaptive fine-tuning achieved 40–60% fewer fine-tuning runs vs. fixed schedules

LIMITATIONS & PIVOT

- Single architecture: Wav2Vec2 CTC
- Single dataset: Edinburgh Noisy Speech
- Bespoke code without reusable RL hooks
- Needed general frameworks and infrastructure for broader applicability

This led directly to our Spring 2026 pivot: we surveyed RL tooling and fine-tuning frameworks to generalize SASL beyond our custom implementation.

Baseline results from Fall 2025 COMS 6998 course project report.

Why We Pivoted: From System to Survey

01

THE REALIZATION

SASL prototype worked, but scaling required robust infrastructure

Needed: continual fine-tuning, RL objectives, and adaptive scheduling

Existing toolkits only partially supported these requirements

02

THE BROADER PROBLEM

Many teams building self-adaptive STT face identical infrastructure choices

Options: NeMo, ESPnet, VERL, TRL, RLlib, etc.

No clear guidance on which frameworks fit which use cases

No systematic comparison of RL capabilities across speech models

03

OUR DECISION

Step back and systematically map the RL tooling landscape

Contribute experiments that stress-test NeMo and ESPnet as RL substrates

Provide practitioners with a framework-agnostic survey and recommendations

Research Questions Driving the Survey

Three core research questions that structure the investigation:

1

How do NeMo and ESPnet structure supervised and parameter-efficient fine-tuning for long-lived STT deployments?

Explores: Training pipelines, PEFT mechanisms (LoRA, adapters), data handling, evaluation protocols

Why it matters: Understanding existing infrastructure is the foundation for adding RL

2

To what extent can RL or RLHF frameworks be repurposed to drive self-adaptive STT?

Explores: Compatibility of LM-focused RL frameworks with speech models, reward signal design, training loop integration

Why it matters: Determines whether we can leverage existing RL tooling or need custom solutions

3

What is missing if one wants a unified self-learning STT stack?

Explores: Gaps in frameworks, missing abstractions, infrastructure requirements

Why it matters: Identifies the design requirements for production-grade self-adaptive STT

STT Architecture Families: Strengths & Weaknesses

CTC

Encoder-only

Strengths: Simple, fast, low latency

Weaknesses: Weak language modeling, sensitive to vocabulary shift

Example: Wav2Vec2

RNN-T

Transducers

Strengths: Streaming with internal language model

Weaknesses: Coupled encoder/decoder, costly training

Example: Conformer-based transducers

Attention Encoder-Decoder & Hybrid CTC-Attention

Strengths: Best accuracy, global context

Weaknesses: Slower, not naturally streaming

Example: ESPnet-style models

CONNECTION TO FINE-TUNING

- Architecture determines where PEFT and RL signals can attach
- Architecture affects risk of catastrophic forgetting during adaptation
- Different architectures require different fine-tuning strategies

RL and RLHF Frameworks: Landscape Overview

LM-Focused RLHF and RLVR Stacks

Frameworks: VERL, OpenRLHF, DeepSpeed Chat, TRL, TRLX, RL4LMs

Design: Assume text token policies and reward models over natural language preferences

Limitation: Built for language models, not speech-specific objectives

Relevance: Can be adapted for STT but require custom reward signals

Speech LM Alignment Stacks

Frameworks: Align SLM, SpeechLM

Design: Apply preference-based learning to discrete speech tokens for speech-conditioned generation

Limitation: Optimized for speech generation, not WER-optimized transcription

Relevance: Closest to STT but designed for a different task: generation vs. recognition

Generic RL Libraries

Frameworks: Ray RLlib

Design: Algorithm-agnostic environments, policies, and replay buffers

Limitation: No ASR-specific components

Strength: Maximum flexibility for custom implementations

Relevance: Requires building STT-specific wrappers and reward functions

NeMo and ESPnet at a Glance

NVIDIA NeMo

- **Encoders:** Conformer and FastConformer
- **Model:** stt_en_conformer_ctc_medium
- **Parameters:** ~30M parameters (18 layers, d_model=256)
- **Tokenizer:** 1,024 SentencePiece BPE subword tokens
- **Decoders:** CTC and RNN-T
- **Data pipeline:** Manifest-based
- **Training:** PyTorch Lightning trainers
- **PEFT support:** Strong support for LoRA, adapters, and IA3 (Infused Adapter by Inhibiting Activations)
- **Key strength:** Comprehensive parameter-efficient fine-tuning via model transforms

ESPnet 2

- **Base checkpoint:** pyf98/librispeech_conformer
- **Architecture:** 12-layer Conformer encoder + 6-layer Transformer decoder, pretrained on LibriSpeech 960h
- **CTC weight:** 0.3 in the hybrid CTC-Attention loss
- **Frozen layers:** Bottom 6 encoder blocks are frozen during fine-tuning
- **Architectures:** Broad catalog (Conformer, Transformer, Branchformer, RNN-T, hybrid CTC-Attention)
- **Scripts:** Recipe-driven approach
- **Evaluation:** Unified evaluation framework
- **Fine-tuning control:** Flexible freezing or scheduler control
- **Logging:** JSONL prediction logs convenient for reward computation
- **Key strength:** Flexibility and reward-friendly output format

Conceptual STT ↔️ RL Mapping and Gaps

THE MAPPING

STT systems already contain the core structure of RL — they just do not expose it in RL-native terms.

Data loaders and manifests

→ Environments producing observations

ASR encoder or decoder

→ Policy that outputs token sequences

WER or CER metrics

→ Reward signals

STT pipelines already define observation inputs, action generation, and performance feedback — the same conceptual building blocks used in RL.

These gaps motivated our experimental focus: we needed to bridge STT frameworks and RL tooling by building explicit reward interfaces and trajectory representations.

THE GAPS

⚠ **Gap 1: Reward Interface**

NeMo and ESPnet treat WER/CER as diagnostic metrics, not first-class reward interfaces.

RL stacks expect reward functions to be easily callable and differentiable.

Result: No clean way to feed WER/CER back into RL training loops.

⚠ **Gap 2: Decoding and Trajectories**

RL stacks have no native notion of CTC decoding or streaming speech trajectories.

Speech models use specialized decoding such as beam search and language model fusion.

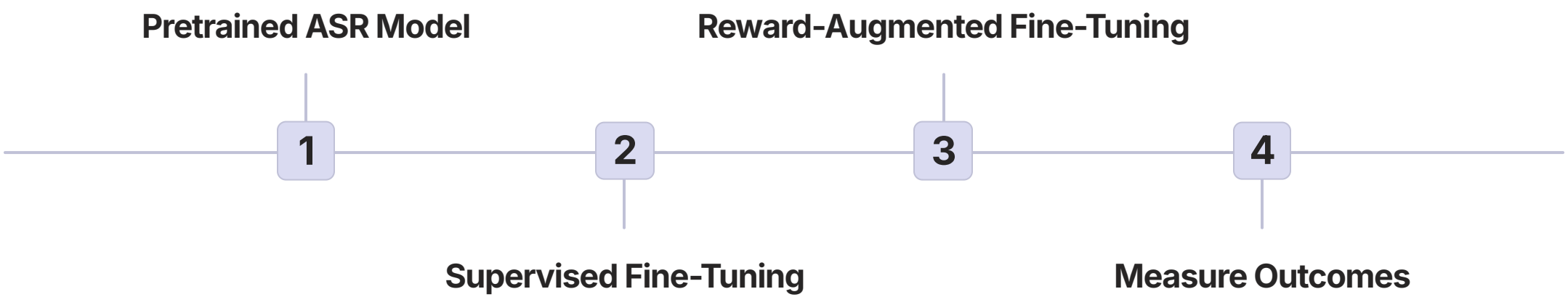
RL frameworks assume discrete token sequences, not acoustic feature streams.

Result: Mismatch between how speech models generate outputs and what RL expects.

Shared Experimental Design for NeMo and ESPnet

High Level Goals

- Evaluate NeMo and ESPnet as substrates for long-lived STT adaptation in the SASL self-learning system, using the same AfriSpeech clinical domain and LibriSpeech forgetting proxy.
- Study how a two-stage supervised then reward-augmented pipeline behaves across both frameworks under matched reward definitions.
- Stress test whether existing fine-tuning infrastructure can be extended with WER-based rewards while keeping training stable and reproducible.



This two-stage pattern (SFT → RL) is applied identically in structure across NeMo and ESPnet, with framework-specific hyperparameters made explicit later.

Datasets and Splits Used Across Both Frameworks

Dataset	Role	Size	Domain and Usage
AfriSpeech Clinical	Primary adaptation dataset for supervised and reward-augmented training.	**~200 hours total** (67,577 clips, ~11 sec avg, ~60% clinical domain)	African English clinical conversations; used to drive both Stage 1 supervised adaptation and Stage 2 reward-augmented updates.
VoxPopuli	Secondary adaptation domain for NeMo RL experiments; mixed into ESPnet SFT training data.	**~543 hours** (~400,000 utterances, full English train split)	English parliamentary speech drawn from European Parliament proceedings. Naturally occurring speech. Acoustically cleaner and closer to LibriSpeech than AfriSpeech clinical.
LibriSpeech	Catastrophic forgetting proxy and secondary evaluation set for both frameworks.	**~5.4 hours** of clean read English speech (used as eval subset only)	Clean, well-resourced English; opposite acoustic domain from clinical African English, used to detect degradation or "negative forgetting" after RL.

Evaluation Protocol and Metrics

Metrics

- Standard: Word Error Rate (**WER**) and Character Error Rate (**CER**).
- Domain specific: Entity level WER (**EWER**) over clinical lexicons and Domain Term F1.
- **EWER (Entity WER):** WER computed only on reference words that appear in the clinical domain lexicon — measures accuracy on specialized vocabulary.
- **Domain Term F1:** Token-level precision, recall, and F1 restricted to domain vocabulary occurrences — measures whether clinical terms are correctly captured in the output.

Statistical Testing

- Bootstrap p-values (1,000 iterations, two-sided, paired at utterance level) comparing SFT vs RL runs on AfriSpeech validation and VoxPopuli validation.
- A single, shared evaluation harness plus explicit p-values is what makes cross-framework comparisons defensible.

Domain Term F1 and EWER use the AfriSpeech clinical lexicon and are not computed for VoxPopuli or LibriSpeech. Extension to VoxPopuli domain metrics is future work.

Reward Signals

Both NeMo and ESPnet use simple, WER-based rewards to reweight the CTC-style loss, without changing model architectures. The goal is to isolate framework differences rather than experiment with complex RL algorithms.

Reward Definitions

- **Minimum WER Reward (MWER)**: For each utterance i, define:

$$r_i = \max(0, 1 - \mathrm{WER}(\mathrm{ref}_i, \mathrm{hyp}_i))$$

  on a 0–1 scale, so lower WER gives higher reward.

- **Weighted WER Reward (WWER)**: Compute a weighted Levenshtein distance where each reference word gets weight 3.0 if it is a domain term and 1.0 otherwise; deletions and substitutions of domain terms use weight 3.0, insertions always cost 1.0. Define WWER-based reward as:

$$r_i = \max(0, 1 - \mathrm{WWER}(\mathrm{ref}_i, \mathrm{hyp}_i))$$

  on a 0–1 scale.

- **Neutral Fallback Reward**: When an utterance is too long (e.g., > 25 seconds) or skipped due to decoding overhead, assign:

$$r_i = 0.5$$

  as a neutral default.

Objective Function

Reweighted CTC Objective (reweight_ctc)

For each sample i in a batch of size B, let ℓ_i be the per-sample CTC loss and r_i ∈ [0,1] the reward. Define a weight:

$$w_i = 1 + \alpha(1 - r_i)$$

where α is the reward weight (NeMo uses α = 0.05; ESPnet uses an equivalent RL weight of 0.02 in its blended objective). The reweighted loss is:

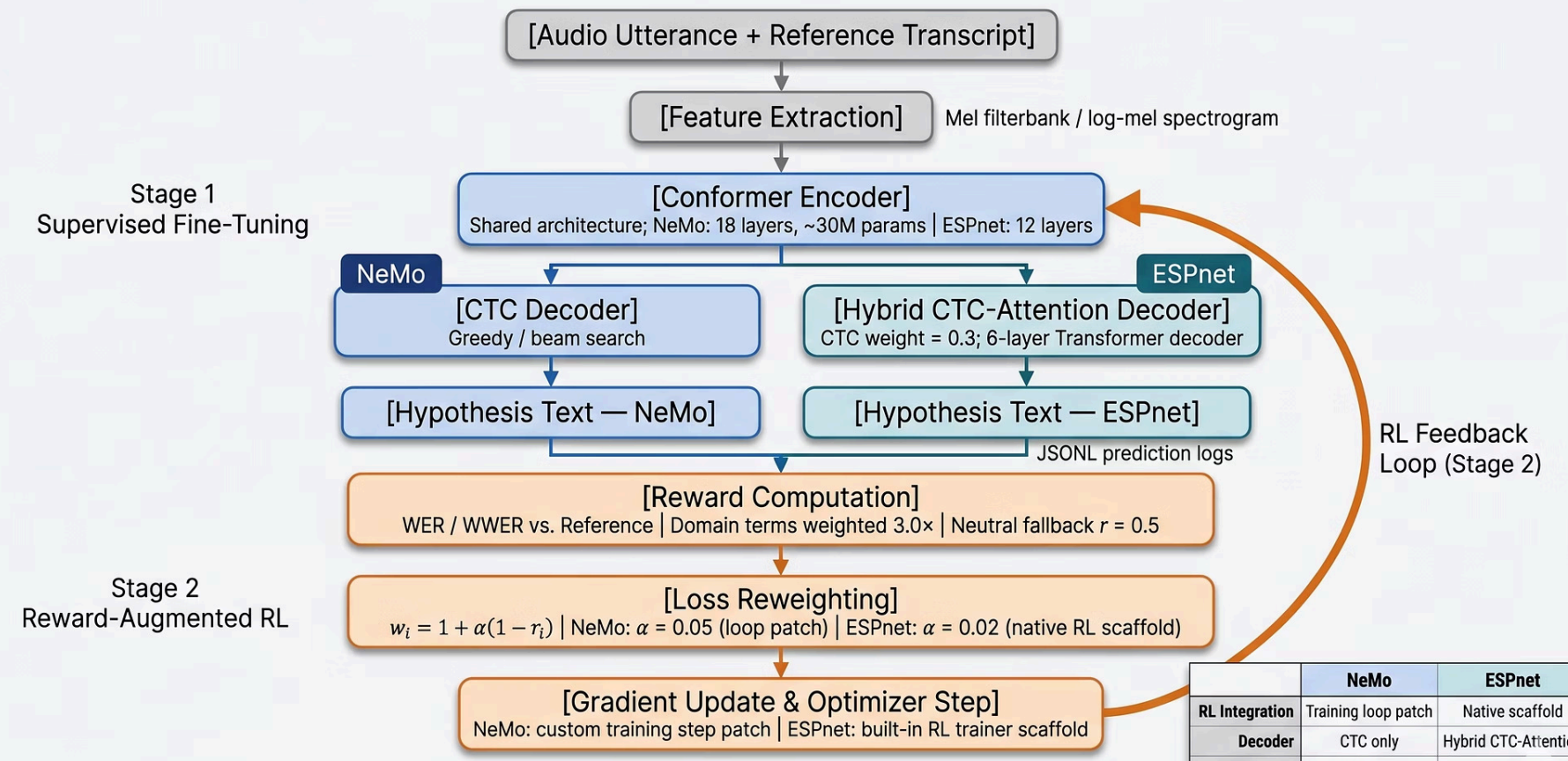
$$L = \frac{1}{B} \sum_i w_i \cdot \ell_i$$

so low-reward (high-error) examples get higher weight and thus stronger gradient updates.

Framework-specific note: NeMo applies reweight_ctc directly to a pure CTC loss with a smaller Stage Two learning rate (1×10^{-5}) and 2 RL epochs; ESPnet blends the same style reward weighting into a hybrid CTC-Attention objective with RL weight 0.02 for 2 RL epochs.

> The reward machinery is intentionally simple and identical across frameworks so that observed differences can be attributed to architecture and tooling, not reward design.

How NeMo and ESPnet Integrate Reward Signals



	NeMo	ESPnet
RL Integration	Training loop patch	Native scaffold
Decoder	CTC only	Hybrid CTC-Attention
RL Weight (α)	0.05	0.02
Key Strength	PEFT (LoRA, IA3)	JSONL reward logs

Reward definitions (MWER, WWER) are identical across both frameworks. Architecture and loop integration differ.

NeMo Key Results

Dataset	Stage	WER (%)	CER (%)	EWER (%)	Domain P	Domain R	Domain F1
AfriSpeech	Zero-shot	57.88	25.87	19.97	0.884	0.857	0.858
	SFT	45.95	**14.19**	20.27	0.842	0.867	0.847
	RL (MWER)	**45.89**	14.21	**19.82**	0.847	0.878	0.855
	RL (WWER)	45.92	14.23	18.92	0.857	0.885	0.864
VoxPopuli	SFT	16.01	5.83	6.74	—	—	—
	RL (MWER)	16.00	**5.82**	**6.60**	—	—	—
	RL (WWER)	**15.98**	5.83	6.94	—	—	—

Domain P, Domain R, and Domain F1 use the AfriSpeech clinical lexicon and are not applicable to VoxPopuli. Extension to a VoxPopuli domain lexicon is future work.

Key finding: MWER and WWER RL runs produce similar WER to SFT (within 0.06 absolute percentage points), while improving domain entity recognition (EWER) and domain F1 scores.

STATISTICAL SIGNIFICANCE (BOOTSTRAP P-VALUES)

AfriSpeech SFT vs MWER RL: p = 0.512

AfriSpeech SFT vs WWER RL: p = 0.609

VoxPopuli SFT vs MWER RL: p = 0.650

VoxPopuli SFT vs WWER RL: p = 0.501

None reach $p < 0.05$ — improvements are real but small relative to utterance-level variance. Single seed, 2-epoch RL runs; multiple seeds and longer training are needed to establish significance.

NeMo Catastrophic Forgetting Proxy: LibriSpeech Results

Training Domain	Stage	WER (%)	CER (%)
AfriSpeech	SFT	**10.44**	**4.09**
	RL (MWER)	10.56	4.15
	RL (WWER)	10.65	4.17
VoxPopuli	SFT	**6.225**	**2.263**
	RL (MWER)	6.317	2.268
	RL (WWER)	6.335	2.278

Note: These are forgetting proxy measurements on LibriSpeech dev-clean (the original pretraining domain). AfriSpeech-trained models show mild forgetting (~0.2 WER points max); VoxPopuli-trained models show even less (~0.1 WER points), consistent with VoxPopuli being acoustically closer to LibriSpeech.

ESPnet Key Results

Framework	Dataset	Stage	WER (%)	CER (%)	Entity WER (%)	Domain F1
ESPnet	AfriSpeech	SFT	66.43	29.25	33.55	0.755
ESPnet	AfriSpeech	RL (WWER)	64.09	26.89	30.02	0.790
ESPnet	LibriSpeech dev-clean	SFT	5.13	2.19	6.25	0.958
ESPnet	LibriSpeech dev-clean	RL (WWER)	4.74	1.95	6.25	0.958

KEY FINDING:

RL (WWER) delivers a 2.34 absolute percentage point WER reduction over SFT (66.43% → 64.09%) and a 3.53pp EWER improvement (33.55% → 30.02%), demonstrating that reward-augmented training successfully targets clinical vocabulary. Domain F1 improves from 0.755 to 0.790.

ESPnet: Catastrophic Forgetting Proxy — LibriSpeech Results

Metric	SFT Baseline	After RL (WWER)	Change
WER	5.13%	4.74%	**↓ 0.39 pp**
CER	2.19%	1.95%	**↓ 0.24 pp**
Entity WER	6.25%	6.25%	— unchanged
Domain F1	0.958	0.958	— unchanged

KEY FINDING

- WWER reward had negligible negative effect on clean English (negative forgetting)
- RL did not overfit to the clinical domain
- Carefully weighted reward-augmented fine-tuning improves domain-specific metrics without harming general performance

NeMo vs ESPnet Results and Takeaways

↓ 11.93pp WER

57.88% → 45.95%

NeMo: zero-shot → SFT improvement
on AfriSpeech clinical

↓ 2.34pp WER

66.43% → 64.09%

ESPnet: RL improvement over SFT
on AfriSpeech clinical

↓ 3.53pp EWER

33.55% → 30.02%

Best entity-level gain
with ESPnet on AfriSpeech

What We Learned

- **SFT does the heavy lifting:** Supervised fine-tuning delivers 10–12pp WER reduction; RL adds at most 2.34pp on top — but reliably improves domain entity metrics even when WER gains are marginal.
- **Domain rewards work as designed:** WWER RL improves Domain F1 in both frameworks (NeMo: 0.847→0.864; ESPnet: 0.755→0.790) — the domain-weighted signal successfully shifts the model toward clinical terminology.
- **No catastrophic forgetting:** NeMo LibriSpeech WER increases by at most +0.21pp; ESPnet actually *improves* by −0.39pp (negative forgetting).
- **Gains are not statistically significant** (all $p > 0.50$ on 2-epoch single-seed runs) — real but require longer experiments to confirm.

Key Patterns: What the Numbers Show

SECTION 1 — RL improves domain metrics more reliably than overall WER

- NeMo AfriSpeech:
  - WER gain 0.03–0.06pp
  - EWER gain up to −1.35pp
  - Domain F1: 0.847 → 0.864
- ESPnet AfriSpeech:
  - WER gain −2.34pp
  - EWER gain −3.53pp
  - Domain F1: 0.755 → 0.790
- VoxPopuli:
  - WER gain 0.01–0.03pp — same modest trend holds outside clinical domain
- AfriSpeech held-out test:
  - RL shows slight WER regression (+0.15–0.32pp) but EWER still improves
  - entity-level gain is the more robust finding

Interpretation: RL helps the model focus on domain-specific terminology without dramatic WER shifts

SECTION 2 — Catastrophic forgetting is negligible

- NeMo: LibriSpeech WER up only +0.21pp after RL
- ESPnet: LibriSpeech WER improves −0.39pp (negative forgetting)
- Conservative RL weights (α = 0.02–0.05), 2 epochs — sufficient to prevent degradation

Interpretation: Conservative RL weights and short training prevent catastrophic forgetting

SECTION 3 — Results not yet statistically significant

- All p-values > 0.50:
  - AfriSpeech MWER p=0.512, WWER p=0.609;
  - VoxPopuli MWER p=0.650, WWER p=0.501
- Single seed, 2 RL epochs — multiple seeds and longer training needed

Interpretation: Improvements are real but require larger-scale experiments to confirm

Next Steps: Roadmap for Self-Adaptive STT

1. Harden the Experiments

Timeline: Q2–Q3 2026

- Run 3–5 seeds to establish statistical significance and variance estimates
- Extend RL training to 5–10 epochs for longer adaptation horizons
- Add LLM-based reward scoring: Med-Gemma or BioMistral for medical domain (full pipeline ~6 days; deferred as future work)
- Publish results with error bars and significance tests

2. Extend to New Domains

Timeline: Q3–Q4 2026

- VoxPopuli-only ESPnet recipes (diverse, multilingual domains)
- Test on noisy corpora (CHiME, JSALT) to probe generalization
- Cross-domain experiments: train on one domain, evaluate on another

3. Build an STT–RL Alignment Layer

Timeline: Q4 2026–Q1 2027

- Design a thin gym-like wrapper exposing NeMo/ESPnet as RL environments (reset, step, reward, done)
- Integrate with VERL or RLlib for algorithm-agnostic training (PPO, A3C)
- Release as open-source

Step 1 validates → Step 2 generalizes → Step 3 standardizes. Together these address the key gaps and move toward a unified, production-ready self-adaptive STT platform.

Reflections: From System to Survey and Back

Building vs. Surveying

Key learning:

- Started with a working SASL system
- Realized scaling required knowing the broader tooling landscape
- Hybrid approach: use the system as a concrete testbed for the survey

RL is an Infrastructure Problem

Key learning:

- NeMo and ESPnet lack first-class RL support — and even when you build it, SFT on good domain data delivers 5× more WER improvement than RL on top
- The real work is in reward APIs, environment abstractions, and continual learning tools

Thank You!

APPENDIX

Code, Paper, and Resources

GITHUB REPOSITORIES

- NeMo RL Experiments: https://github.com/shivangi221b/ASR-Fine-Tuning-Modules-and-RL-Adaptation-Analysis
- ESPnet RL Fork: https://github.com/GAInTheHouse/espnet/tree/feature/espnet2-rl-training-docs

DOCUMENTATION

- Experiments and Results: `experiments_and_results.md` (in NeMo repo)
- Detailed methodology, hyperparameters, and full results tables

PAPER AND PRESENTATION

- Paper PDF: https://github.com/shivangi221b/ASR-Fine-Tuning-Modules-and-RL-Adaptation-Analysis/reports/Spring 2026 Research Course/Research Presentation.pdf
- Presentation Video: https://github.com/shivangi221b/ASR-Fine-Tuning-Modules-and-RL-Adaptation-Analysis/reports/Spring 2026 Research Course/Research Presentation Video.mp4

CONTACT AND COLLABORATION

- All code is open-source and available for community use and extension

This work is part of the SASL Spring 2026 research project at Columbia University.

Shared Experimental Design for NeMo and ESPnet

High Level Goals

- Evaluate NeMo and ESPnet as substrates for long-lived STT adaptation in the SASL self-learning system, using the same AfriSpeech clinical domain and LibriSpeech forgetting proxy.
- Study how a two-stage supervised then reward-augmented pipeline behaves across both frameworks under matched reward definitions.
- Stress test whether existing fine-tuning infrastructure can be extended with WER-based rewards while keeping training stable and reproducible.

Concrete Two-Stage Pipeline

Step 1: Pretrained ASR Model

Start from a pretrained CTC-based ASR model in each framework (NeMo: 30M parameter Conformer CTC; ESPnet: Conformer encoder + Transformer decoder hybrid CTC-Attention).

Step 2: Stage 1 - Supervised Fine-Tuning

Fine-tune on AfriSpeech clinical with standard CTC or hybrid CTC-Attention loss. NeMo: 5 epochs, AdamW, learning rate 1×10^{-4}. ESPnet: supervised recipe over AfriSpeech + VoxPopuli + LibriSpeech 5k subset, hybrid CTC-Attention loss with CTC weight 0.3 and frozen lower encoder layers.

Step 3: Stage 2 - Reward-Augmented Fine-Tuning

Add non-differentiable rewards derived from WER and weighted WER (WWER) on decoded hypotheses. NeMo: uses the reweight_ctc objective with reward weight 0.05 for 2 additional epochs at learning rate 1×10^{-5}. ESPnet: uses the same reweight_ctc style objective with RL weight 0.02 for 2 RL epochs.

Step 4: Measure Outcomes

In-domain: AfriSpeech validation and test WER, CER, entity WER, and Domain Term F1. Stability: LibriSpeech dev-clean WER as a catastrophic forgetting proxy.

This two-stage pattern (SFT → RL) is applied identically in structure across NeMo and ESPnet, with framework-specific hyperparameters made explicit here.

Survey Scope: What We Investigated

STT FINE-TUNING FRAMEWORKS

Focus: Representative toolchains for STT fine-tuning

NVIDIA NeMo

Why: Production-focused, with strong PEFT support

ESPnet

Why: Research-friendly, with flexible architecture options

RL AND RLHF TOOLING

LM-focused

VERL, OpenRLHF, TRL, TRLX, RL4LMs, DeepSpeed Chat

Speech-specific

Align SLM, SpeechLM

Generic RL

Ray RLlib

Why this range: Understand what exists across different abstraction levels and domains

Quantitative Comparison Summary: NeMo

Framework	Dataset	Stage	WER (%)	CER (%)	Entity WER (%)	Domain F1
NeMo	AfriSpeech	Zero-shot	57.88	25.87	19.97	0.858
NeMo	AfriSpeech	SFT	45.95	14.19	20.27	0.847
NeMo	AfriSpeech	RL (MWER)	45.89	14.21	19.82	0.855
NeMo	AfriSpeech	RL (WWER)	45.92	14.23	18.92	0.864
NeMo	VoxPopuli	SFT	16.01	5.83	6.74	—
NeMo	VoxPopuli	RL (MWER)	16.00	5.82	6.60	—
NeMo	VoxPopuli	RL (WWER)	15.98	5.83	6.94	—
NeMo	AfriSpeech Test	SFT	50.68	17.13	23.28	—
NeMo	AfriSpeech Test	RL (MWER)	50.83	17.17	22.83	—
NeMo	AfriSpeech Test	RL (WWER)	51.00	17.18	22.87	—

Note: Validation results show stronger domain adaptation on AfriSpeech than on VoxPopuli, with the largest gains appearing in entity-level and domain-specific metrics. Test set results on AfriSpeech (3,508 utterances) indicate that RL maintains some entity-level improvements, but does not meaningfully reduce overall WER. Overall, the results suggest RL is most effective as a domain adaptation strategy on validation data rather than a broad test-time WER improvement.

Training times:

AfriSpeech: SFT: 12,243 s (~3.4 hours); RL (MWER): 4,382 s (~1.2 hours); RL (WWER): 4,502 s (~1.25 hours)

Domain F1 uses the AfriSpeech clinical lexicon and is not applicable to VoxPopuli. It was not computed in the post-hoc AfriSpeech test evaluation run.

NeMo Architecture and Training Details

Model and Encoders

- Conformer-based CTC model used for English ASR
- Around thirty million parameters with eighteen encoder layers and moderate hidden dimension
- SentencePiece BPE tokenizer with around one thousand subword units

Training Pipeline Features

- Manifest-based data pipeline integrated with NeMo training scripts
- Strong built-in support for parameter-efficient fine-tuning including LoRA, adapters, and IA3-style transforms
- PyTorch Lightning trainers for scalable experiments

These features make NeMo a good representative for production-oriented STT fine-tuning with strong PEFT support.

ESPnet Architecture and Training Details

Model and Encoders

- Conformer encoder with around twelve layers paired with a Transformer decoder pretrained on LibriSpeech 960 hours
- Hybrid CTC plus attention loss with a moderate CTC weight
- Flexible catalog of architectures including Conformer, Transformer, RNN-T, and hybrid models

Training Pipeline Features

- Recipe-driven scripts that standardize data preparation and training
- Unified evaluation framework and JSONL prediction logs that make it easy to compute custom rewards
- Fine-tuning control through layer freezing and scheduler configuration

ESPnet is more research-oriented and flexible relative to NeMo, which will feed into the explicit comparison slides.

WHY ESPnet

- Positions ESPnet as a **complementary testbed** rather than a replacement
- **NeMo** focused on CTC-only models with strong PEFT support
- **ESPnet** offers richer architectural diversity and RL-aware infrastructure
- Goal: show RL-augmented STT is **not NeMo-specific** and can generalize across frameworks

ESPNET STRENGTHS FOR RL-AUGMENTED STT

- Rich ASR recipes: **Hybrid CTC-Attention** models, more expressive than CTC-only
- Fine-grained control: flexible initialization and freezing strategies
- Reward-friendly logging: **JSONL prediction logs** enable easy per-utterance reward computation
- RL-aware infrastructure: built-in scaffold blends **cross-entropy** and **policy gradient** terms
- Architectural diversity: **Conformer**, **Transformer**, **Branchformer**, and **RNN-T** options

ESPnet Experiment Design and Architecture

EXPERIMENT DESIGN & KEY DIFFERENCE

Model: pyf98/librispeech_conformer

Architecture: 12-layer Conformer encoder + 6-layer Transformer decoder

Pretrained on: LibriSpeech 960h

Hybrid loss CTC weight: 0.3

During SFT: bottom 6 encoder blocks frozen

Training data:

- AfriSpeech clinical: ~58,000 utterances
- VoxPopuli: ~400,000 utterances
- LibriSpeech subset: 5,000 utterances

Rationale: richer pretraining data than the NeMo experiment, testing generalization across datasets

FRAMEWORK COMPARISON

- **NeMo:** patched the training loop to add reward signals to the CTC loss
- **ESPnet:** uses an existing RL-aware trainer scaffold, requiring less custom code
- **Implication:** RL infrastructure for STT can be built into frameworks from the start

NeMo and ESPnet Configuration Side by Side

This slide focuses on configuration details that directly affect how easy it is to bolt RL-style objectives onto an existing STT stack.

Feature	NeMo	ESPnet
Base Model Family and Architecture	Conformer-based CTC-only encoder (18 layers, ~30M parameters, d_model=256)	Conformer encoder (12 layers) + Transformer decoder (6 layers), hybrid CTC-Attention
Tokenization and Output Units	SentencePiece BPE with ~1000 subword units	SentencePiece BPE with flexible vocabulary size, supports multiple tokenization schemes
Type of Decoder and Streaming Support	CTC greedy and beam search decoders; limited streaming support	Attention-based and hybrid CTC-Attention decoders; native streaming support via RNN-T variants
Parameter-Efficient Fine-Tuning	Built-in LoRA, adapters, and IA3 transforms; integrated with training loop	Layer freezing and scheduler configuration; less integrated PEFT infrastructure
How Reward Signals Are Injected	Custom training loop patch that reweights CTC loss per utterance based on reward	Extends existing RL-aware trainer scaffold; leverages built-in infrastructure for reward blending
Additional Implementation Details for RL	Manifest-based data pipeline; tight integration with PyTorch Lightning; reward computation happens in training step	JSONL prediction logs for offline reward computation; recipe-driven scripts; more flexibility for custom reward functions

NeMo vs. ESPnet as RL Substrates: Comparative Analysis

ARCHITECTURE AND TRAINING

- **NeMo:** ~30M parameter Conformer CTC-only (stt_en_conformer_ctc_medium, 18 layers, d_model=256)
- Training data: AfriSpeech clinical only
- Trainer: PyTorch Lightning with custom training step patches
- Approach: Minimal architectural changes, maximum code reuse
- **ESPnet:** ~100M parameter Conformer with joint CTC-Attention decoder
- Training data: Combined AfriSpeech + VoxPopuli + LibriSpeech 5k
- Trainer: ESPnet's native trainer with RL blends integrated at model level
- Approach: Richer architecture, more diverse pretraining

GAPS IN BOTH FRAMEWORKS

- PPO (Proximal Policy Optimization) infrastructure
- Replay buffers for off-policy learning
- Streaming-aware controllers for online adaptation
- Native support for non-differentiable reward signals (WER, CER)
- Unified RL-STT APIs

RL INTEGRATION STRATEGY

- **NeMo:** "Hack" approach—inject reward into CTC loss via reweighting
- Implementation: Patched training step that computes rewards and scales losses
- Pros: Minimal framework changes, works with existing infrastructure
- Cons: Ad-hoc, not generalizable to other loss functions or architectures
- **ESPnet:** Scaffold approach—cleaner separation of supervised and REINFORCE-style terms
- Implementation: Blends cross-entropy and policy gradient terms at model level
- Pros: More principled, better separation of concerns
- Cons: Still lacks PPO-style infrastructure, replay buffers, streaming-aware controllers

IMPLICATIONS FOR PRACTITIONERS

- NeMo: Good for quick prototyping, works with existing pipelines
- ESPnet: Better for research, more flexible architecture
- Neither is a complete RL-STT platform yet
- Both require custom code for production-grade self-adaptive STT

Design Requirements for Self-Adaptive STT Infrastructure

Speech-Native Environment Abstraction

Treat ASR models as policies acting on audio streams, not just text token sequences.

- **Offline mode:** Process batches of audio files with full context
- **Streaming mode:** Handle real-time audio with latency constraints
- **Observation space:** Raw audio or acoustic features, not just text
- **Action space:** Token sequences with confidence scores and timing information
- **Trajectory representation:** Audio → intermediate representations → decoded text → metrics

Why it matters: Current RL frameworks assume discrete token policies; speech has continuous acoustic streams. Streaming and offline modes have different requirements, and standard abstractions enable algorithm reuse across STT architectures.

Standardized Reward Interfaces

Define clean APIs for STT-specific reward signals that RL algorithms can plug into.

- **WER:** Standard metric, non-differentiable
- **CER:** Finer-grained error signal
- **Entity WER:** Domain-specific terminology accuracy
- **Diarization error:** Speaker attribution accuracy
- **Semantic scores:** LLM-based correctness or relevance signals

Why it matters: Different domains need different reward signals. Reward computation is expensive, so standard APIs enable caching and batching while cleanly decoupling reward design from RL implementation.

First-Class Continual Learning Tools

Build infrastructure for long-lived, adaptive STT systems, not one-off fine-tuning.

- **Replay buffers:** Store and sample from past utterances for off-policy learning
- **Drift-aware sampling:** Prioritize recent or out-of-distribution data
- **Adaptive schedulers:** Integrate compute, latency, and forgetting into retraining decisions
- **Monitoring and rollback:** Track metrics and revert to previous checkpoints if needed

Why it matters: Production STT systems face continuous drift. Principled tools are needed to decide when and how to retrain, and cost-aware scheduling prevents wasteful compute.

Integration Vision

1. Environment abstraction enables RL algorithms to interact with STT models
2. Reward interfaces let algorithms optimize domain-specific objectives
3. Continual learning tools keep the system adaptive and cost-efficient over time

Together, these requirements bridge the gap between research RL frameworks and production self-adaptive STT systems.

Infrastructure Gaps: Critical Challenges Identified

Pain Point 1: Training Loop Ownership Conflicts

NeMo and ESPnet own the training loop; RL frameworks expect to own it. Patching vs. scaffolding creates friction and maintenance burden. No clean abstraction for "insert reward signal here".

Impact: Requires custom code for each framework

Implication: STT frameworks need first-class RL support, not patches

Pain Point 2: Lack of Standard Reward APIs

WER and CER are non-differentiable, computed post-hoc from decoded outputs. No native support in NeMo or ESPnet for reward functions. RL frameworks expect differentiable reward signals.

Impact: Practitioners must build custom reward computation and integration

Implication: Standard reward APIs for speech metrics (WER, CER, entity-level) are essential

Pain Point 3: Missing RL Infrastructure

No replay buffers for off-policy learning. No scheduling support for adaptive RL weight or learning rate. No streaming-aware controllers for online adaptation. No standard way to handle long sequences or variable-length utterances.

Impact: Full-featured self-adaptive STT requires significant engineering

Implication: Hybrid approaches (supervised + RL) need better abstractions

Concrete Next Steps: Roadmap for Self-Adaptive STT

1. Harden the AfriSpeech pipelines

Current state: Single seed, 2 epochs of RL, limited reward signals

Next steps:

- Run multiple seeds (3–5) to establish statistical significance and variance estimates
- Extend RL training to 5–10 epochs to explore longer adaptation horizons
- Implement richer reward modes:
- LLM-based semantic correctness scores using Med-Gemma (Google) or BioMistral (HuggingFace) for medical domain reward modeling (full pipeline ~6 days; deferred as future work)
- Confidence-weighted WER for selective fine-tuning
- Multi-objective rewards balancing WER, entity accuracy, and latency
- Publish results with error bars and significance tests

Timeline: Q2–Q3 2026

Impact: Validate that RL gains are robust and reproducible

2. Extend to other datasets and domains

Current state: AfriSpeech clinical only

Next steps:

- VoxPopuli-only recipes in ESPnet (multilingual, diverse domains)
- Additional noisy corpora (CHiME, JSALT, etc.) to test generalization
- Cross-domain experiments: train on one domain, test on another
- Benchmark against domain-specific baselines (e.g., fine-tuned models)

Timeline: Q3–Q4 2026

Impact: Demonstrate that RL-augmented STT generalizes beyond clinical speech

3. Prototype an STT RL alignment layer

Current state: Custom code for NeMo and ESPnet

Next steps:

- Design a thin wrapper that exposes NeMo/ESPnet models as RL environments
- Implement standard gym-like interface (reset, step, reward, done)
- Integrate with VERL or RLlib to enable algorithm-agnostic RL training
- Test with PPO, A3C, or other standard RL algorithms
- Document the interface and release as open-source

Timeline: Q4 2026–Q1 2027

Impact: Enable broader adoption and algorithm experimentation

BROADER VISION

- Step 1 validates the approach with statistical rigor
- Step 2 demonstrates generalization across domains and languages
- Step 3 bridges STT frameworks and RL tooling with a standard interface

Together, these steps address the key gaps identified in the survey and move toward a unified, production-ready self-adaptive STT platform.